OpenSearch Ingestion

This document helps IBM field teams quickly identify which ingestion approach and IBM solution best fits a customer's use case when working with OpenSearch.

Context

To unlock the value of the data an organization holds with OpenSearch, the data must first be captured, organized, and prepared so it can be searched effectively.

In practice, this means building an ingestion pipeline that transforms raw data into searchable content. The resulting data should support semantic search, keyword search, and the ability to refine results using domain knowledge or information about the user making the query.

Purpose

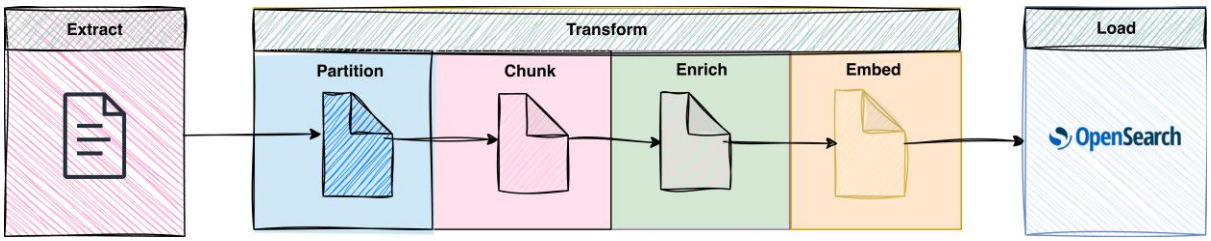
This document provides high-level guidance for field teams on selecting ingestion strategies to propose to customers so they can meet their use cases and take advantage of the full capabilities of OpenSearch.

The goal is to build a reliable ingestion pipeline that transforms enterprise information into high-quality searchable data in OpenSearch, enriched with embeddings and metadata when required.

High Level Data Flow

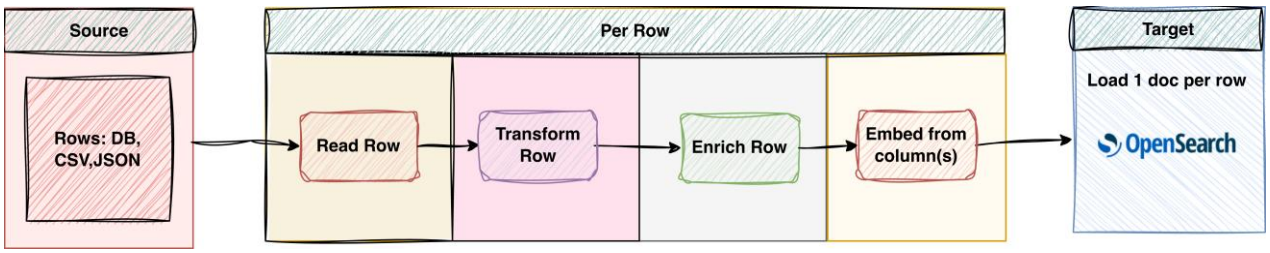
Unstructured Data Flow:

At a high level, ingesting unstructured data (documents, files) into a search or RAG system follows an **Extract → Transform → Load** pattern. The transform stage usually includes **partition**, **chunk**, **enrich**, and **embed** steps.



Structured data flow:

For structured data (databases, CSV, JSON), ingestion is **row-by-row**: each row (or record) is processed as one unit and becomes one searchable document. There is no chunking. The pipeline extracts rows, shapes each row into a search schema, optionally enriches it, and optionally **embeds** from one or more text columns (e.g. one vector from **title + description**) before loading. This pattern fits enterprise search over catalogues, tickets, assets, or any tabular/JSON source.



Ingestion Solutions Overview

Customer may opt to bring their own pipeline to ingest data into OpenSearch but if they would like guidance on the products that IBM recommends the following solutions are commonly used to build ingestion pipelines for OpenSearch:

watsonx.data Integration	Watsonx.data integration features a unified control plane that allows you to integrate both **structured** and **unstructured** data by using a mix of styles—including batch, real-time streaming and replication—so you are not wasting time and money toggling between tools.
Unstructured.io	IBM partner solution designed to process **a wide range of document formats**, extracting and preparing content for search pipelines.
watsonx.data- Spark Engine	Managed Apache Spark runtime used to build large-scale ingestion pipelines and custom data processing steps without operating a Spark cluster.
IBM Docling	IBM open-source library for document parsing and chunking, used to extract and structure content from documents so it can be efficiently processed and indexed in ingestion pipelines.

***Note**: Apache Spark may appear in several parts of this guide. It can run either as the managed watsonx.data Spark Engine or as a customer-managed Spark cluster depending on the deployment model.*

At a high level: two approaches

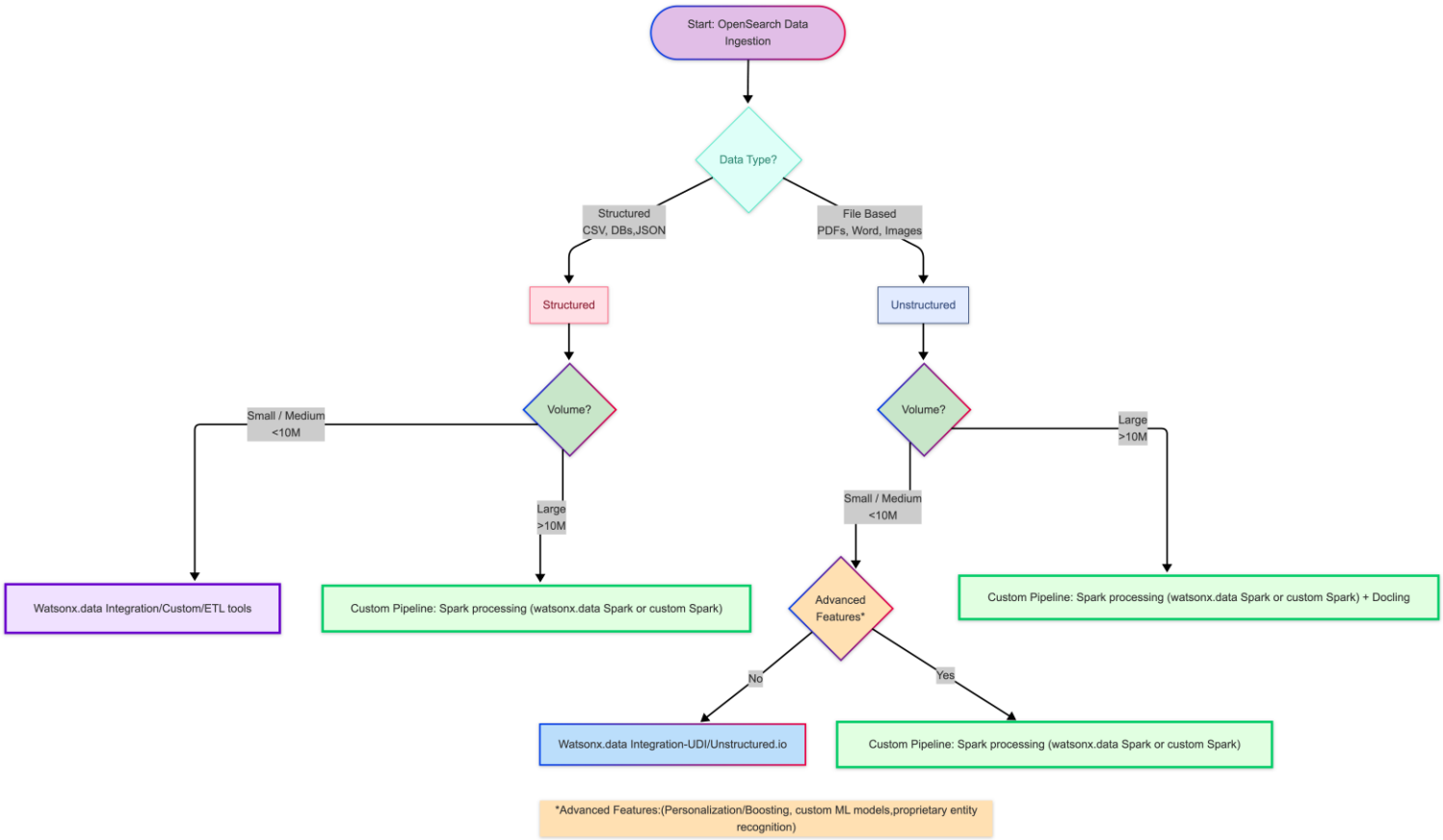
- **Option A – Managed tools first**
  Prefer managed ingestion where it fits. Use **IBM watsonx.data Integration** for structured batch (e.g. DataStage) and for file-based ingestion (e.g. UDI). **Unstructured.io** (IBM partner) is another option for file-based data.

- **Option B – Custom pipeline**: Use when volume, complexity, or control go beyond what the managed tools offer. You can design the ingestion flow (for example using Apache Spark for distributed processing and IBM Docling for document parsing). Spark processing can run either on the **managed watsonx.data Spark Engine** or on a **customer-managed Spark** cluster.

Decision tree

Use the flowchart below to identify the most appropriate approach based on **data type and volume**. The diagram applies to **batch ingestion**. For streaming structured data (such as logs, CDC, or event streams), see the streaming section below.

Volume units: For structured data, volume is measured in **records (rows)**. For file-based data, volume is measured in **documents** (one file = one document). The thresholds in the diagram are guidelines; actual limits depend on the size and complexity of the records or documents.



How to help guide the decision

Step 1: Data type → Step 2: Volume

Path A: Structured (CSV, DB, JSON)

What it is: Records from databases, CSV files, or JSON.
Volume: Count **records** (rows). Each record is usually one searchable unit (one embedding per record). Embeddings are generated using an embedding model.

Volume (records)	Recommended option
Small / Medium <10M	**Watsonx.data Integration** (or another ETL tool if the customer is not on the IBM stack)
Large >10M	**watsonx.data Spark Engine / customer managed Spark Cluster**

Path B: File based (PDFs, Word, Images)

What it is: Documents such as PDF, DOCX, or images.
Volume: Count **documents** (one file = one document).

The pipeline **chunks** each document (a chunk can be several lines, a paragraph, or a section); each chunk is one searchable unit (different from structured, where one row = one unit). Chunk embeddings are generated using an embedding model.

Images: Need OCR or similar before chunking; Docling and Unstructured.io support this.

Step 3: Advanced Features

For **small or medium volumes (<10M documents)**, determine whether **advanced features** are required. Advanced features refer to use cases where ingestion requires **custom enrichment, domain-specific ML models, or additional metadata used for ranking, filtering, or personalization during search**.

If advanced features are required, use a **custom solution** (for example **Spark + Docling**). If not, use **watsonx.data integration or Unstructured.io**.

Volume (documents)	Advanced Features*?	Recommended option
Small / Medium <10M	No	**Watsonx.data Integration or Unstructured.io**
Small / Medium <10M	Yes	**Custom** (Spark-based pipeline (watsonx.data Spark Engine or customer-managed Spark cluster) + Docling)
Large >10M	(N/A)	**Custom** (Spark-based pipeline (watsonx.data Spark Engine or customer-managed Spark cluster) + Docling)

What do watsonx.data integration and Unstructured.io provide?

These tools are designed to prepare **document-based data in chunks**, rather than ingesting data line-by-line as in structured ETL.

Both tools can chunk documents, generate embeddings (using an embedding service such as **IBM watsonx.ai**), extract metadata, support keyword indexing at the chunk

level, and extract entities (for example as arrays in the document or chunk metadata). This is typically sufficient for **semantic search and keyword search**.

However, they do not support **custom fields for filtering or boosting**, or more advanced ingestion features such as **personalization, custom ML models, or proprietary entity recognition**. For those requirements, use a **custom solution**.

watsonx.data integration or Unstructured.io: which one?

The choice depends mainly on **file sources and governance requirements**. See the solution comparison for more details.

- **Prefer watsonx.data Integration** when enterprise features are required (e.g., audit, lineage, PII handling) or when data sources align with the watsonx.data integration connector model.

- **Prefer Unstructured.io** when broader file format support or more flexible connectors or deployment options are needed.

Building a custom pipeline

Use this section when a managed ingestion tool is not sufficient and a custom pipeline is required.
Custom pipelines often rely on **Apache Spark** for scalable data processing. Spark can run either on the **watsonx.data Spark Engine (managed runtime)** or on a **customer-managed Spark cluster**, depending on the architecture and operational requirements.

Custom pipelines typically fall into the following categories:

- **File-based data (small/medium volume) with Advanced Features***
  Advanced features include personalization, custom ML models, or proprietary entity recognition. Use a custom pipeline such as **Spark + Docling**. Docling handles document parsing and chunking, while Spark provides scalable processing.

- **Structured data (large volume)**
  Use **Apache Spark** for large-scale processing. If the pipeline requires workflow scheduling or dependency management, add an orchestration tool such as **Apache Airflow**.

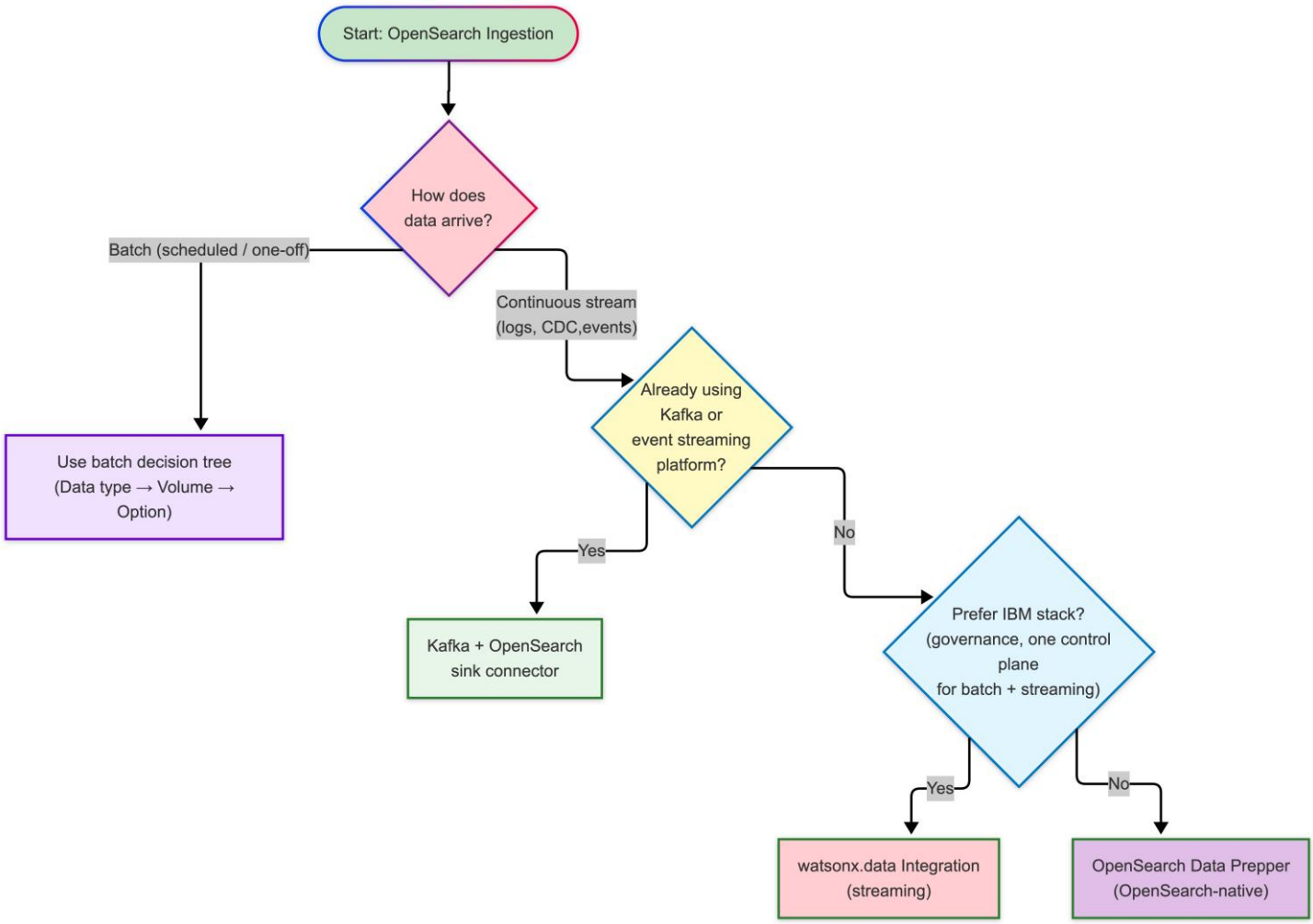
- **File-based data (large volume)**
  Use **Docling** for document parsing and chunking, and **Spark** for distributed processing and scale. This approach also enables additional processing such as keyword extraction, metadata enrichment, entity recognition, and custom fields for filtering or boosting in OpenSearch.

Streaming Ingestion

Streaming ingestion is used for data that arrives continuously (logs, CDC, events) and needs to be indexed in OpenSearch quickly. Instead of scheduled batch loads, each event is processed as it arrives. The pipeline can apply transformations and enrichment before indexing, and optionally call a model service (e.g., watsonx.ai) to generate embeddings.

If Kafka already exists, Kafka with an OpenSearch sink connector is usually the simplest path, with transformations and embeddings generated upstream (Kafka Streams, Flink, Spark Streaming, or a custom service).

If Kafka is not part of the architecture and the organization prefers the IBM stack, watsonx.data Integration can be used to build and manage the pipeline with IBM governance and integration capabilities. If the IBM stack is not required, OpenSearch Data Prepper provides an OpenSearch-native streaming ingestion option.



Overall Solutions

IBM also offers overarching solutions that include ingestion as part of an end-to-end value proposition.

Q&A RAG Accelerator

https://www.ibm.com/docs/en/watsonx/saas?topic=accelerators-qa-rag-accelerator

OpenRAG

https://ibm.seismic.com/Link/Content/DC7QfhMDg2qhG8MRR8H8p3DpdJ73#/?anchorId=ea5a7820-5e1f-48dd-82dd-d6838e7ca55f